

Efficient Transaction Traceability for Auditable Privacy-Preserving Ledgers

Elli Androulaki, Angelo De Caro, Kaoutar Elkhiyaoui, Rebekah Mercer, and
Elina van Kempen

IBM Research Europe – Zurich

Abstract. Privacy-preserving distributed ledgers form the basis of transaction systems where users can interact without revealing their identities or transaction details. Permissionless systems demonstrate strong anonymity guarantees, but regulated and institutional settings introduce additional requirements. Authorised parties must be able to efficiently trace transactions for auditing purposes without undermining privacy of the system as a whole. Existing approaches provide verifiable tracing but introduce limitations, either preventing users from forming transactions in parallel, having high computation cost for either the user or auditor, or granting auditors unrestricted access to user secrets.

We present a framework for efficient tracing for privacy-preserving transaction systems that avoids global counters and restricts auditor access to user secrets. We formalise our security and privacy requirements with an ideal functionality and present a black-box construction based on pseudorandom functions and anonymous credentials that we prove secure via simulation. We give two concrete instantiations: one using hash-based PRFs and zk-SNARKs, and one using algebraic PRFs compatible with bilinear-group proof systems. An implementation and experimental evaluation demonstrate that our approach enables efficient tracing while preserving user privacy and transaction parallelism.

1 Introduction

Distributed ledger technologies (DLTs) are increasingly deployed in settings where financial transparency and regulatory compliance must coexist with strong user privacy guarantees. While early permissionless systems such as Bitcoin provide only limited pseudonymity through one-time public keys [20], transaction graphs and account balances remain globally visible and become linkable over time [7]. Ethereum and similar platforms offer comparable pseudonymous models. Even when user identities are not explicitly revealed, transaction metadata and graph structure can enable de-anonymisation in practice. Privacy-focused cryptocurrencies such as Zcash aim to provide stronger transaction anonymity by hiding senders, receivers, and amounts within a large anonymity set [6, 18]. However, these systems are designed primarily for open, permissionless environments and do not address the privacy requirements of regulated financial systems.

As DLTs are adopted for regulated assets, central bank digital currencies (CBDCs), and institutional settlement systems privacy requirements evolve [17, 16]. In these settings, users are typically enrolled by a registration authority and issued credentials attesting to their eligibility in the system. Users must be able to transact privately, without exposing their identities, transaction contents, or transaction creation patterns, while still enabling regulatory oversight under defined conditions [2, 4, 23, 1]. This gives rise to a privacy tension which must be resolved if private transaction systems are to be put into practice. Systems must provide strong anonymity against the public and other users or unauthorised parties, yet enable auditing mechanisms that satisfy legal and compliance requirements.

One important oversight mechanism in such systems is transaction tracing. Given a specific user and with appropriate authorisation, an auditor should be able to identify all transactions associated with that user, without learning information about unrelated users. Tracing should not expose the activity of other users.

Existing approaches to tracing in privacy-preserving ledgers exhibit important limitations. A common simple suggestion is for systems to embed encrypted identifiers in each transaction, which doesn't allow for selective deanonymisation but instead requires ledger-wide processing to trace a single user. Others rely on sequential counter-based tags, which enable recomputation of user-linked tags but impose ordering constraints on transaction generation. In high-throughput or distributed environments, counter-based dependencies prevent parallel transaction construction.

Solution Overview. We propose a tracing mechanism tailored to regulated privacy-preserving ledger systems that avoids both encrypted user identifiers and sequential counter-based tag derivation. Instead of maintaining a globally incrementing counter, users derive transaction tags from a pseudorandom function (PRF) evaluated on independently chosen indices from a bounded range $[0, max)$. Users prove in zero knowledge that each tag is correctly computed with respect to their secret key and that the index lies within the permitted range.

Transactions are organised into time-bound epochs in order to give auditing powers a natural expiration point. At the beginning of each epoch, users derive an epoch-specific tracing key used to generate tags within that epoch only. Tracing is therefore scoped to a defined time window and revoked at each epoch boundary. This design enables parallel transaction generation, eliminates sequential dependencies, and restricts auditor visibility to authorised periods.

Concrete Instantiations and Evaluation. We instantiate our framework in two concrete ways suitable for deployment in real systems. The first uses hash-based PRFs and zk-SNARKs to prove correct tag generation and range membership. The second employs algebraic PRFs over bilinear groups compatible with sigma protocols yielding more efficient zero-knowledge proofs.

We implement both instantiations and evaluate their performance, measuring proof generation time, verification time, and communication overhead. Our

results demonstrate that the proposed tracing mechanism is practical for high-throughput privacy-preserving payment systems and compatible with modern zero-knowledge infrastructures.

Contributions. We summarise our contributions as follows:

- We formalise privacy-preserving traceable ledger systems using a simulation-based security definition that captures both user anonymity and regulated tracing requirements.
- We present a black-box construction based on pseudorandom functions and anonymous credentials that enables efficient, non-interactive, epoch-bound tracing without sequential counters or encrypted identifiers.
- We prove security of the construction against a static, polynomial-time adversary.
- We provide two concrete instantiations compatible with widely deployed zero-knowledge proof systems.
- We implement and benchmark both constructions, demonstrating their practicality for real-world privacy-preserving ledger deployments.

1.1 Related Work

Prior work on regulatory oversight in privacy-preserving payment systems typically supports either anonymity revocation or tracing. Anonymity revocation allows an auditor to identify the user responsible for a specific transaction, while tracing allows the auditor to identify all transactions associated with a given user. Tracing can be implemented by applying anonymity revocation to each transaction individually, but this incurs computational cost linear in the number of transactions and reveals information about unrelated users.

Anonymity revocation. Many systems attach a verifiable encryption or otherwise hidden user identifier to each transaction [8, 9, 3, 5, 11, 10]. The identifier can be recovered by an auditor holding a secret key, enabling selective deanonymization of individual transactions. In such systems, the auditor can selectively decrypt a specific transaction to recover the sender’s identity. Tracing here requires scanning and decrypting all transactions on the ledger. This introduces computational overhead linear in ledger size and exposes non-targeted activity to the auditor. These systems are said to support anonymity revocation or identity recovery rather than efficient user-level tracing.

Tracing via sequential tags. Several systems derive transaction tags from a user secret and a monotonically increasing counter [19, 25]. Counter-based techniques for tracking assets, and coin lineage rather than user activity have also been proposed [13]. This approach generally enables efficient tracing by recomputing tags associated with a target user. The use of sequential counters introduces inherent limits on parallel transaction construction: transactions must be generated in order, and missed or reused counters must be addressed. This limits concurrency

and reduces practicality in high-throughput or distributed environments. Peredi is a CBDC-based construction that uses sequential tags and zero-knowledge proofs to establish correct progression [19]. Tracing is performed by an auditor that iteratively recomputes the sequence of tags until no further matches are found on the ledger. PARScoin follows a similar design in an account-based model aimed at stable coin systems [25].

Sequential-tag designs introduce inherent sequentiality: transactions must be generated in order, missed counters must be carefully handled, and concurrent generation of multiple transactions from the same user requires coordination. In distributed or high-throughput settings, this sequential dependency can reduce practicality and complicate secure element implementations.

Auditable privacy systems. Other systems provide auditability through interactive protocols or global consistency checks. ZKLedger enables auditors to verify transaction correctness but requires interaction and incurs overhead linear in the number of participants [21]. Privacy-focused cryptocurrencies such as Monero, Zcash, and Quisquis provide strong anonymity guarantees but do not directly support efficient tracing [22, 6, 12]. Retrofitting tracing typically requires additional interaction, user cooperation, or linear-time analysis [24]. Platypus is a CBDC proposal that incorporates regulatory policy enforcement based on balance limits or asset-backing proofs but does not provide mechanisms for efficient per-user tracing across arbitrary transactions [26].

Our contribution. We present a tracing mechanism that avoids both encrypted user identifiers and sequential counters. Our construction enables efficient, non-interactive tracing while supporting parallel transaction generation. Tracing is epoch-bound, allowing auditors to identify all transactions associated with a user within a given time period without imposing sequential constraints or requiring interaction with other users. To our knowledge, this is the first construction that achieves efficient tracing without relying on either encrypted identifiers or sequential tag generation.

2 Privacy-preserving Traceable Ledgers

2.1 Security requirements

We consider a transaction system with anonymous transaction submission and user-controlled, per-epoch tracing by auditors. Users should be able to submit transactions without revealing their identities, while retaining the ability to selectively authorize auditors to trace their activity in specific epochs. Once granted, tracing should be complete and non-repudiable, and users should be able to revoke tracing permissions across epochs. Finally, the system should support efficient tracing and concurrent transaction generation. The system must satisfy the following properties:

- Anonymity. For all auditors and all epochs, an auditor’s view reveals no information about which users participated in transactions during that epoch, beyond what is explicitly revealed by granted tracing permissions.
- Non-repudiation. All transactions accepted by the ledger within a given epoch on behalf of a given user are traceable by any auditor holding valid tracing permission for that user and epoch.
- Revocation. Users must have fine-grained control over which auditors may trace their transactions. Auditor permissions are reset with each new epoch.
- Efficiency. Tracing should not have computational dependence on total number of transactions. Users should be able to perform parallel tag generation to enable concurrent payments.

2.2 Security formalization

We capture the security of our system using a real/ideal world simulation-based definition. Rather than defining games for each of the security properties above, we specify an ideal functionality $\mathcal{F}_{\text{Trace}}$ that captures the functionality of a transaction system with anonymous transactions and per-epoch tracing. A concrete protocol is secure if any real execution can be simulated given only the information explicitly revealed by the ideal functionality.

The ideal functionality described in Figure 1 models a transaction system where users can register, submit transactions, and allow approved auditors to trace their activity for a given epoch. It also exposes an interface for the global system epoch to be reset. We briefly describe the interfaces below:

- Register.** A user can register only once. If registration succeeds, the system records that the user holds valid credentials, which are required for transaction issuance.
- Submit.** A registered user can submit up to max transactions during the current epoch. The system logs each transaction, recording the user and epoch involved.
- Grant.** A user may grant a specific auditor the right to trace all of their transactions in a specific epoch. Once both parties confirm, the system marks the auditor as permitted for the given user and epoch.
- Trace.** An auditor who has been granted permission for a user and epoch can request the list of that user’s transactions for that epoch. The system returns exactly that set.
- MoveEpoch.** When the system moves to a new epoch, auditors tracing permissions are all cleared.

We now explain how the functionality captures the security requirements.

Anonymity. For any auditor A , any epoch e , and any two users U_0 and U_1 , if $\text{perm}[(A, U_b, e)] = 0$ for $b \in \{0, 1\}$, then the view of A in epoch e is independent of whether U_0 or U_1 issued any particular transaction. The functionality internally records the association between users and their transactions in $\text{txns}[(U, e)]$, but this information is revealed only through successful **Trace** calls by auditors

Register. On input Register from user U, send (Register, U) to RA and wait for confirmation. On receiving confirmation, do the following: 1. If $\text{reg}[U] \neq \perp$, abort. 2. Send (Register, U) to $\mathcal{S}$ and wait for the go-ahead. On receiving the go-ahead, continue. 3. Set $\text{reg}[U] = \text{cred}$. 4. Return (RegisterEnd, cred) to U.	for confirmation. On receiving confirmation, do the following: 1. Send (Grant, A, U, e) to $\mathcal{S}$. If $\mathcal{S}$ returns <i>reject</i>, abort. 2. Set $\text{perm}[(A, U, e)] = 1$. 3. Return GrantEnd to U, A, and $\mathcal{S}$.
Submit. On input (Submit, tx, e) from U, do the following: 1. If $e \neq e$, abort. 2. If $\text{reg}[U] = \perp$, abort. 3. Send (Submit, U, tx, e) to $\mathcal{S}$, and wait for the go-ahead. 4. Add tx to the set $\text{txns}[(U, e)]$. 5. Return SubmitEnd to U and $\mathcal{S}$.	Trace. On input (Trace, U, e) from A, do the following: 1. If $\text{perm}[(A, U, e)] = 0$, abort. 2. Send (Trace, A, U, e) to $\mathcal{S}$, and wait for the go-ahead. 3. Set $T = \text{txns}[(U, e)]$. 4. Return (TraceEnd, T) to A and to $\mathcal{S}$.
Grant. On input (Grant, A, e) from U, send (Grant, U, e) to A and wait	MoveEpoch. On input MoveEpoch: 1. Let $e \leftarrow e + 1$. 2. Send (MoveEpochEnd, e) to $\mathcal{S}$.

Fig. 1. Ideal functionality $\mathcal{F}_{\text{Trace}}$ for tracing with epochs.

who have been explicitly granted permission. Since **Trace** aborts whenever the required permission is absent, no auditor or other party learns any information linking transactions to users without user consent.

Non-repudiation. For any user U and epoch e , once a transaction tx is successfully submitted, the functionality records tx in $\text{txns}[(U, e)]$. No interface allows transactions to be modified or removed. Consequently, any auditor with valid permission for (U, e) learns the complete and unaltered set of transactions issued by U in that epoch via a **Trace** call, ensuring that users cannot deny or repudiate their submitted transactions.

Revocation. Auditor permissions are explicitly scoped to a single epoch. When the functionality advances to a new epoch via **MoveEpoch**, all existing permissions are cleared. As a result, auditors can trace transactions only for epochs in which the user has explicitly granted permission.

Efficiency. The ideal functionality reflects the intended efficiency requirements by providing direct access to the stored transaction sets for tracing queries, without requiring iteration over unrelated users or transactions. This captures

the goal that tracing complexity depends only on the traced user and epoch, and not on the total number of transactions in the system.

Security of the concrete protocol is shown by constructing a simulator that, for any adversary interacting with the real protocol, can produce an indistinguishable execution using only the interfaces of $\mathcal{F}_{\text{Trace}}$.

3 Building Blocks

3.1 Commitments

A commitment scheme is defined by a triple of algorithms:

- $\text{ComKeygen}(1^\kappa) \rightarrow CK$: on input security parameter κ , ComKeygen returns a public commitment key CK .
- $\text{Commit}(CK, m) \rightarrow (\rho, C)$: on input commitment key CK and message m , Commit outputs an (optional) secret opening hint ρ and a commitment C .
- $\text{Open}(CK, m, \rho, C) \rightarrow 0/1$: on input commitment key CK , message m , hint ρ , and C , Open returns 0 or 1.

A commitment scheme is secure if it satisfies the properties of correctness and binding, and should additionally satisfy the property of hiding. Definitions of the security properties of commitments are given in Appendix 9.2.

3.2 Pseudorandom Functions (PRF)

A family of *deterministic* functions $\mathbf{F}(k, \cdot) : \{0, 1\}^l \rightarrow \{0, 1\}^\kappa$, indexed by seed $k \in \{0, 1\}^\kappa$, is pseudorandom if it is:

- **efficiently computable**: given $(k, x) \in \{0, 1\}^\kappa \times \{0, 1\}^l$, $\mathbf{F}(k, x)$ can be computed in polynomial time;
- **pseudorandom**: it is computationally infeasible for a distinguisher to tell the output of $\mathbf{F}(k, \cdot)$, on input x provided by the distinguisher, apart from that of a uniformly random function, even when given access to a polynomial-number of outputs $y_i = \mathbf{F}(k, x_i)$ for inputs x_i that she adaptively chooses.

3.3 Digital Signatures (DS)

A digital signature scheme involves algorithms:

- $\text{DSKeygen}(\kappa) \rightarrow (SK, PK)$: on input security parameter κ , DSKeygen outputs a pair of secret key SK and public key PK .
- $\text{DSSign}(SK, m) \rightarrow \sigma$: on input of secret key SK and message $m \in \{0, 1\}^*$, DSSign outputs signature σ .
- $\text{DSVerify}(PK, m, \sigma) \rightarrow 0/1$: on input public key PK , message m , and signature σ , DSVerify outputs 0 or 1.

A DS scheme is said to be **existentially unforgeable** if for any adversary that adaptively chooses a polynomial number of messages m_i and is given access to signatures $\sigma_i : \text{DSVerify}(PK, m_i, \sigma_i) \rightarrow 1$ for some public key PK , she cannot output a pair $(m, \sigma) : m \neq m_i \wedge \text{DSVerify}(PK, m, \sigma) \rightarrow 1$.

3.4 Proofs of Knowledge

Informally, zero-knowledge proofs of knowledge allow a prover to convince a verifier that they know a secret witness w for a public statement x , without revealing w [14].

More formally, a zero-knowledge proof system is defined with respect to a language $\mathcal{L}_{\mathcal{R}}$ and an associated binary relation $\mathcal{R}$. An instance $x \in \{0, 1\}^*$ is said to belong to the language $\mathcal{L}_{\mathcal{R}}$ if and only if there exists a witness $w \in \{0, 1\}^*$ such that $(x, w) \in \mathcal{R}$. The value x is called the statement and w is called the witness for x .

In a proof of knowledge for the relation $\mathcal{R}$, the prover holds a witness w such that $(x, w) \in \mathcal{R}$, or equivalently, $\mathcal{R}(w, x) = 1$. The verifier knows only the relation $\mathcal{R}$ and public statement x . The goal of the protocol is for the prover to convince the verifier knowledge of a valid witness with $(x, w) \in \mathcal{R}$, without revealing any additional information about w .

Zero-knowledge is formalised by existence of a simulator that can efficiently generate transcripts indistinguishable from real protocol executions without access to a witness.

Non-Interactive Zero-Knowledge Proofs (NIZKs). A NIZK scheme consists of the following algorithms:

- $c \xleftarrow{\$} \mathcal{G}(\mathcal{R})$: on input relation $\mathcal{R}$, output common reference string for the relation $\mathcal{R}$.
- $\pi \xleftarrow{\$} \mathcal{P}(c, w, x)$: on input c, w, x such that $\mathcal{R}(w, x) = 1$, output proof π that w satisfies $\mathcal{R}$ for x .
- $\{0, 1\} \leftarrow \mathcal{V}(c, x, \pi)$: on input reference string c , public statement x , and proof π , outputs 0 or 1.

To capture the necessary security properties of NIZKs, we define the following algorithms:

- $(c, \tau) \xleftarrow{\$} \text{NIZK.SimGen}(\mathcal{R})$: on input a relation $\mathcal{R}$, output a common reference string c and trapdoor τ . The distribution of c is computationally indistinguishable from $\mathcal{G}(\mathcal{R})$. The trapdoor τ allows simulating proofs and extracting witnesses from adversarial proofs.
- $\pi \xleftarrow{\$} \text{NIZK.SimProve}(c, x, \tau, m)$: on input (c, x, τ) and optional message m , output a simulated proof π . The distribution of π is computationally indistinguishable from a real proof produced by $\mathcal{P}(c, w, x)$.
- $\hat{w} \leftarrow \mathcal{X}(c, x, \tau, \pi, m)$: on input (c, x, τ, π, m) , output an extracted witness $\hat{w}$. If $\mathcal{V}(c, x, \pi) = 1$, then $(x, \hat{w}) \in \mathcal{R}$.

Correctness. For all $(w, x) \in \mathcal{R}$ and all messages m , the following holds:

$$\Pr \left[\mathcal{V}(c, x, \pi) = 1 \mid c \xleftarrow{\$} \mathcal{G}(\mathcal{R}), \pi \xleftarrow{\$} \mathcal{P}(c, w, x, m) \right] = 1 - \varepsilon(\kappa).$$

Simulatability. There exists a polynomial-time simulator ($\text{NIZK.SimGen}, \text{NIZK.SimProve}$) such that for all PPT adversaries $\mathcal{A}$, the following holds:

$$\left| \Pr \left[b = 1 \mid (\mathfrak{c}, \tau) \xleftarrow{\$} \text{NIZK.SimGen}(\mathcal{R}), b \xleftarrow{\$} \mathcal{A}^{\mathcal{S}(\mathfrak{c}, \tau, \cdot)}(s, \mathfrak{c}) \right] - \Pr \left[b = 1 \mid \mathfrak{c} \xleftarrow{\$} \mathcal{G}(\mathcal{R}), b \xleftarrow{\$} \mathcal{A}^{\mathcal{P}(\mathfrak{c}, \cdot)}(s, \mathfrak{c}) \right] \right| \leq \varepsilon(\kappa),$$

the oracle $\mathcal{S}$ receives $(\mathfrak{x}, \mathfrak{w}, m)$, checks $\mathcal{R}(\mathfrak{w}, \mathfrak{x}) = 1$, and returns the following:

$$\pi \xleftarrow{\$} \text{NIZK.SimProve}(\mathfrak{c}, \mathfrak{x}, \tau, m).$$

Extractability. There exists an extractor $\mathcal{X}$ such that for all PPT $\mathcal{A}$ and all auxiliary inputs s :

$$\Pr \left[(\mathfrak{x}, \hat{\mathfrak{w}}) \in \mathcal{R} \vee (\mathfrak{x}, m) \in Q \vee \mathcal{V}(\mathfrak{c}, \mathfrak{x}, \pi) = 0 \mid \begin{array}{l} (\mathfrak{c}, \tau) \xleftarrow{\$} \text{NIZK.SimGen}(\mathcal{R}), \\ (\mathfrak{x}, m, \pi) \xleftarrow{\$} \mathcal{A}^{\mathcal{S}(\mathfrak{c}, \tau, \cdot)}(s, \mathfrak{c}), \\ \hat{\mathfrak{w}} \leftarrow \mathcal{X}(\mathfrak{c}, \mathfrak{x}, \tau, \pi, m) \end{array} \right] = 1 - \varepsilon(\kappa), \quad (1)$$

where Q is the set of prior valid queries answered by $\mathcal{S}$, i.e., all $(\mathfrak{x}, m)$ for which $\mathcal{A}$ previously submitted a witness and received a simulated proof.

Signatures of Knowledge (SoKs). A signature of knowledge attaches a message m to the non-interactive proof of knowledge, turning the proof into a signature. Signature of knowledge schemes are defined using the following algorithms:

- $pp \xleftarrow{\$} \text{SoKSetup}(\mathcal{R})$: on input relation $\mathcal{R}$, output public parameters pp .
- $\sigma \xleftarrow{\$} \text{SoKSign}(pp, \mathfrak{x}, \mathfrak{w}, m)$: on input public parameters pp , and $\mathfrak{x}, \mathfrak{w}$ such that $\mathcal{R}(\mathfrak{w}, \mathfrak{x}) = 1$, and message m , output a signature of knowledge σ .
- $\{0, 1\} \leftarrow \text{SoKVerify}(pp, \mathfrak{x}, m, \sigma) \rightarrow \{0, 1\}$: on input public parameters pp , $\mathfrak{x}$, message m , and signature σ , output 0 or 1.

The correctness, simulatability, and extractability properties for SoKs are identical to those for NIZKs, with the addition of m . The trapdoor τ allows the simulator to produce signatures for arbitrary messages, and the extractor can recover the witness from any signature produced by an adversary.

4 Black-box Construction

Let $\mathcal{L}$ denote the ledger that processes user transactions. Let $\mathbf{G}$ and $\mathbf{G}'$ be two one-way functions and $\mathbf{F}_E$ and $\mathbf{F}_T$ be two PRF families. Finally, let max be the upper-bound of the number of transactions a user can submit during any given epoch e . An epoch is defined in terms of ledger block height. We namely define an epoch e as the interval between block $e \times l$ and $(e + 1) \times l - 1$, where l is the duration of the epoch.

As a reminder, the protocol is as follows. Each user holds a long-term signing key sk and tag key tk , which are bound to one another by a credential issued by the registration authority. With each epoch, a per-epoch tracing key is computed by each user, using their certified tk and the epoch number e as inputs. Transactions for that epoch are tagged with a PRF evaluated over this epoch key and any number between 0 and max . Users prove correctness of the tag computation via NIZKs. The ledger verifies the proofs. At the time of an audit, the per-epoch user key is shared with the auditor, who can then compute all possible tags for that user and epoch. When the epoch is incremented, a new per-epoch tracing key will be formed by the user, and the auditor will no longer be able to trace that user's transactions.

Setup. During setup, registration authority RA calls $(SK, PK) \leftarrow \text{DSKeygen}(1^\kappa)$ and communicate PK to the ledger operator. The ledger operator calls $CK \leftarrow \text{ComKeygen}(1^\kappa)$ and produces four sets of public parameters:

- $pp_0 \leftarrow \text{SoKSetup}(\mathcal{R}_0)$ for relation $\mathcal{R}_0$ defined for instance and witness defined as follows:

$$\mathbf{x}_0 = (\mathbf{G}, \mathbf{G}', S, T), \quad \mathbf{w}_0 = (sk, tk), \quad (2)$$

such that $\mathcal{R}(\mathbf{x}_0, \mathbf{w}_0) = 1$ if and only if the following holds:

$$S = \mathbf{G}(sk) \quad \wedge \quad T = \mathbf{G}'(tk). \quad (3)$$

- $(\tau_1, pp_1) \leftarrow \mathcal{G}(\mathcal{R}_1)$ for relation $\mathcal{R}_1$ defined for instance and witness defined as follows:

$$\mathbf{x}_1 = (\mathbf{F}_E, CK, C, e, ek), \quad \mathbf{w}_1 = (tk, \rho), \quad (4)$$

such that $\mathcal{R}(\mathbf{x}_1, \mathbf{w}_1) = 1$ if and only if the following holds:

$$\text{Open}(CK, tk, \rho, C) \rightarrow 1 \quad \wedge \quad ek = \mathbf{F}_E(tk, e). \quad (5)$$

- $pp_2 \leftarrow \text{SoKSetup}(\mathcal{R}_2)$ for relation $\mathcal{R}_2$ defined for instance and witness defined as follows:

$$\mathbf{x}_2 = (\mathbf{G}, \mathbf{G}', PK, CK, C), \quad \mathbf{w}_2 = (\sigma, sk, tk, \rho), \quad (6)$$

such that $\mathcal{R}_2(\mathbf{x}_2, \mathbf{w}_2) = 1$ if and only if the following holds:

$$\text{DSVerify}(PK, \mathbf{G}(sk) \parallel \mathbf{G}'(tk), \sigma) \rightarrow 1 \quad \wedge \quad \text{Open}(CK, tk, \rho, C) \rightarrow 1. \quad (7)$$

- $(\tau_3, pp_3) \leftarrow \mathcal{G}(\mathcal{R}_3)$ for relation $\mathcal{R}_3$ defined for instance and witness defined as follows:

$$\mathbf{x}_3 = (\mathbf{F}_E, \mathbf{F}_T, CK, C, e, max, tag), \quad \mathbf{w}_3 = (tk, \rho, ek, \gamma), \quad (8)$$

such that $\mathcal{R}_3(\mathbf{x}_3, \mathbf{w}_3) = 1$ if and only if the following holds:

$$\text{Open}(CK, tk, \rho, C) \rightarrow 1 \quad \wedge \quad ek = \mathbf{F}_E(tk, e) \quad (9)$$

$$\wedge \quad tag = \mathbf{F}_T(ek, \gamma) \quad \wedge \quad 0 \leq \gamma < max. \quad (10)$$

The ledger operator concludes the setup by recording in the ledger entry

$$\mathcal{L}[pp] \leftarrow (\mathbf{G}, \mathbf{G}', \mathbf{F}_E, \mathbf{F}_T, PK, CK, pp_0, pp_1, pp_2, pp_3, max).$$

These parameters reflect the SoK used for initial registration, the NIZK used for proof of correct epoch key computation, the SoKs each user uses to authorize each transaction, and the NIZK used to prove transaction tag computation correctness.

Register. User U randomly selects a signing key sk and a tag key tk , computes $S = \mathbf{G}(sk)$ and $T = \mathbf{G}'(tk)$, and then engages in an interactive protocol with registration authority RA , where she receives a nonce $\mathbf{n}$ and then computes and sends back $\psi_0 \leftarrow \text{SoKSign}(pp_0, \mathbb{x}_0, w_0, \mathbf{n})$, for $(\mathbb{x}_0, w_0)$ as defined in Equation 3.

If RA accepts ψ_0 , then it calls $\text{DSSign}(SK, S \parallel T)$ and sends the resulting signature σ to U . From this point onward, we assume that U uses $\mathbf{G}(sk)$ as her unique identifier in the system. Notably, she uses it to identify herself to auditors.

Grant. In preparation for authorization requests for some epoch e_i , U computes the epoch's tracing key $ek_i = \mathbf{F}_E(tk, e_i)$ and $(C_{0,i}, \rho) \leftarrow \text{Commit}(CK, tk)$, and generates NIZK $\pi_{0,i} \leftarrow \mathcal{P}(pp_1, \mathbb{x}_1, w_1)$, where $\mathbb{x}_1$ and w_1 are defined in Equation 4. An authorization request for some epoch e_i identifies its originating auditor A and contains pair $(e_i, \mathbf{n})$ whereby $\mathbf{n}$ is a random nonce. U grants A the tracing authorization by sending tuple $(ek_i, C_{0,i}, \pi_{0,i}, \psi_{1,i})$ where $\psi_{1,i} \leftarrow \text{SoKSign}(pp_2, \mathbb{x}_2, w_2, \mathbf{n})$ and $(\mathbb{x}_2, w_2)$ are defined as in Equation 7. If $\pi_{0,i}$ and $\psi_{1,i}$ are valid, then A accepts tracing key ek .

Submit. Let e denote the current epoch. U first prepares the payload of her transaction **payload**. Then she selects a new $0 \leq \gamma < max$ that she has not used in this epoch before, and computes $\text{tag}_\gamma = \mathbf{F}_T(ek_i, \gamma)$. Next, she computes commitments $(C_{1,\gamma}, \rho) \leftarrow \text{Commit}(CK, tk)$ and generates $\pi_{1,\gamma} \leftarrow \mathcal{P}(pp_3, \mathbb{x}_3, w_3)$, for $(\mathbb{x}_3, w_3)$ as defined in Equation 8. She concludes by producing $\psi_{2,\gamma} \leftarrow \text{SoKSign}(pp_2, \mathbb{x}_2, w_2, m)$, where $m = (\text{payload}, \text{tag}_\gamma, C_{1,\gamma}, \pi_{1,\gamma})$ and $(\mathbb{x}_2, w_2)$ are defined in Equation 7, and submitting transaction formed as follows to the ledger for validation:

$$\text{tx} = (\text{payload}, \text{tag}_\gamma, C_{1,\gamma}, \pi_{1,\gamma}, \psi_{2,\gamma}). \quad (11)$$

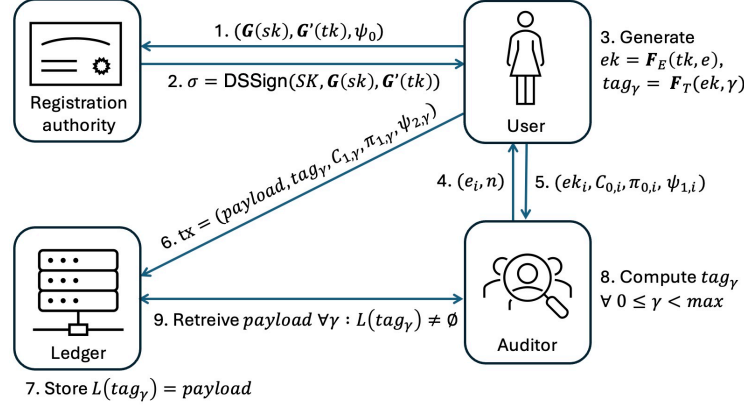
Upon receipt of the transaction, the ledger verifies if tx is valid; namely, it checks if **(1)** tag_γ has never appeared in the ledger before, **(2)** $\pi_{1,\gamma}$ is a valid NIZK for relation $\mathcal{R}_3$, $\psi_{2,\gamma}$ is a valid SoK on message $m = (\text{payload}, \text{tag}_\gamma, C_{1,\gamma}, \pi_{1,\gamma})$ relative to relation $\mathcal{R}_2$ and current epoch e .

If all the checks succeed, the ledger adds entry $\mathcal{L}[\text{tag}_\gamma] \leftarrow \text{payload}$.

Trace. Given key ek_i for epoch e_i , auditor A traces the transactions that U submitted during epoch e_i as follows. $\forall 0 \leq \gamma < max$: it computes $\text{tag}_\gamma^{(i)} = \mathbf{F}_T(ek_i, \gamma)$ and retrieves entry $\mathcal{L}[\text{tag}_\gamma^{(i)}]$ if not empty.

IncrementEpoch. When the ledger reaches block height $(e+1) \times l$, the system participants move to epoch $e+1$.

An overview of the construction is given in Figure 2.

**Fig. 2.** System overview.

Theorem 1. *Assuming existential unforgeability of DS, pseudo-randomness of F_E and F_T , one-wayness of G and G' , and security of NIZK and SoK, the protocol described above securely realizes $\mathcal{F}_{\text{Trace}}$ in the presence of a static, polynomial-time adversary.*

The protocol inherits the security guarantees of $\mathcal{F}_{\text{Trace}}$: anonymity of users, non-repudiation of transactions, and auditor tracing restricted per user and per epoch. The proof of this theorem is given in Appendix 11.

5 Instantiations

We describe two concrete instantiations of the black-box construction. The first uses hash-based PRFs to generate the tags and zk-SNARKs to prove their correctness, and as a result assumes a trusted setup. The second relies on a Bilinear Decisional Diffie-Hellman (BDDH) based PRF and tailored Sigma protocols that enjoy a transparent setup. Both instantiations use Pedersen commitments to instantiate $C \leftarrow \text{Commit}(CK, tk)$, and BBS+ signatures to instantiate $\sigma \leftarrow \text{DSSign}(SK, G(sk) || G'(tk))$. The full Pedersen commitment and BBS+ signature schemes are given in Appendix 9, along with the full description of the BDDH assumption. The choice of BBS+ stems from the ongoing standardization efforts that aim to use them for self-sovereign identity.

5.1 Instantiation using Hash-based PRFs

This instantiation realizes the black-box construction using the BBS+ signature scheme for registration credentials; the Pedersen commitment scheme for commitments; hash-based PRFs for epoch keys and transaction tags; and zk-SNARKs to instantiate the NIZK proof systems $\mathcal{G}(\mathcal{R}_1)$ and $\mathcal{G}(\mathcal{R}_3)$. The SoK

systems $\text{SoKSetup}(\mathcal{R}_0)$ and $\text{SoKSetup}(\mathcal{R}_2)$ are instantiated using Fiat-Shamir proofs of knowledge.

We use independent hash functions $\mathcal{H}_{\text{FS}}$ for Fiat-Shamir challenges, $\mathcal{H}_{\text{E}}$ for epoch key derivation, and $\mathcal{H}_{\text{T}}$ for tags. Let G_1, G_2 be cyclic groups of prime order p with generators $g_1 \in G_1$ and $g_2 \in G_2$.

We instantiate the components as follows:

- The one-way functions $\mathbf{G}$ and $\mathbf{G}'$ are instantiated as

$$\mathbf{G}(sk) = g_1^{sk}, \quad \mathbf{G}'(tk) = g_2^{tk}. \quad (12)$$

- The PRF families $\mathbf{F}_{\text{E}}$ and $\mathbf{F}_{\text{T}}$ are instantiated using independent hash-based PRFs $\mathcal{H}_{\text{E}}$ and $\mathcal{H}_{\text{T}}$:

$$\mathbf{F}_{\text{E}}(tk, e) = \mathcal{H}_{\text{E}}(tk, e), \quad \mathbf{F}_{\text{T}}(ek, \gamma) = \mathcal{H}_{\text{T}}(ek, \gamma). \quad (13)$$

- The signature scheme is instantiated using the BBS+ signature scheme.
- Commitments are instantiated using the Pedersen commitment scheme.
- The NIZK proof systems are instantiated using zk-SNARKs.
- The SoK systems are instantiated using proofs of knowledge, made non-interaction using the Fiat-Shamir heuristic in the random oracle model, using hash function $\mathcal{H}_{\text{FS}}$.

Public parameters. The registration authority generates BBS+ signing keys $(SK, PK) \leftarrow \text{DSKeygen}(1^\kappa)$. The ledger operator generates commitment keys $CK \leftarrow \text{ComKeygen}(1^\kappa)$ and public parameters defined as follows:

- $pp_0 \leftarrow \text{SoKSetup}(\mathcal{R}_0)$ for relation $\mathcal{R}_0$ defined by:

$$S = g_1^{sk} \quad \wedge \quad T = g_2^{tk}.$$

- $(\tau_1, pp_1) \leftarrow \mathcal{G}(\mathcal{R}_1)$ for relation $\mathcal{R}_1$ defined by:

$$\text{Open}(CK, tk, r, C) \rightarrow 1 \quad \wedge \quad ek = \mathcal{H}_{\text{E}}(tk, e).$$

- $pp_2 \leftarrow \text{SoKSetup}(\mathcal{R}_2)$ for relation $\mathcal{R}_2$ defined by:

$$\text{DSVerify}(PK, S \| T, \sigma) \rightarrow 1 \quad \wedge \quad \text{Open}(CK, tk, r, C) \rightarrow 1.$$

- $(\tau_3, pp_3) \leftarrow \mathcal{G}(\mathcal{R}_3)$ for relation $\mathcal{R}_3$ defined by:

$$\begin{aligned} &\text{Open}(CK, tk, r, C) \rightarrow 1 \quad \wedge \quad ek = \mathcal{H}_{\text{E}}(tk, e) \\ &\wedge \quad \text{tag} = \mathcal{H}_{\text{T}}(ek, \gamma) \quad \wedge \quad 0 \leq \gamma < \text{max}. \end{aligned}$$

The ledger records $(PK, CK, pp_0, pp_1, pp_2, pp_3, \text{max})$.

SoK instantiation. We illustrate the instantiation of $\text{SoKSign}(pp_0, \mathbb{x}_0, \mathbb{w}_0, \mathbf{n})$, with $\mathbb{x}_0 = (S, T)$ and $\mathbb{w}_0 = (sk, tk)$. The prover does the following:

- Samples $(\alpha, \beta) \leftarrow \mathbb{Z}_p^2$ and computes $R = g_1^\alpha g_2^\beta$.
- Computes challenge $c = \mathcal{H}_{\text{FS}}(S, T, R, \mathbf{n})$.
- Forms $(a, b) = (\alpha + csk, \beta + ctk)$

The proof is $\psi_0 = (a, b, c)$.

The Register, Grant, Submit, Trace, and IncrementEpoch algorithms are identical to the black-box construction in Section 4, using the concrete primitives and relations given above.

5.2 Instantiation using Algebraic (BDDH-based) PRF

We instantiate the black-box construction of Section 4 using bilinear groups and algebraic PRFs. This yields a transparent setup and efficient pairing-based proofs.

Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ be bilinear groups of prime order p , with generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$.

As in the previous instantiation, BBS+ signatures are used for credentials and Pedersen commitments for commitments. The SoK systems are instantiated using Fiat-Shamir proofs of knowledge.

Epoch and tag PRFs. The epoch key PRF $\mathbf{F}_E$ is instantiated as follows:

$$ek = \mathbf{F}_E(tk, e) := \mathcal{H}_E(e)^{tk} \in \mathbb{G}_1, \quad (14)$$

where $\mathcal{H}_E : \{0, 1\}^* \rightarrow \mathbb{G}_1$ is a hash-to-group function modeled as a random oracle. This satisfies pseudorandom function security under the DDH assumption in $\mathbb{G}_1$. This is captured in Lemma 1 and proven in Appendix 9.5.

With $\gamma \in [0, max)$ with the bit decomposition $(x_0, \dots, x_\ell)$, and generators $G_0, \dots, G_\ell \in \mathbb{G}_2$, the tag PRF $\mathbf{F}_T$ is instantiated as follows:

$$\text{tag} = \mathbf{F}_T(ek, \gamma) = e(ek, \prod_{i=0}^{\ell} G_i^{x_i}). \quad (15)$$

This satisfies pseudorandom function security under the BDDH assumption. This is captured in Lemma 2 and proven in Appendix 9.6.

Public parameters. The registration authority generates BBS+ keys $(SK, PK) \leftarrow \text{DSKeygen}(1^\kappa)$. The ledger operator generates commitment keys $CK \leftarrow \text{ComKeygen}(1^\kappa)$, and samples generators $G_0, \dots, G_\ell \in \mathbb{G}_2$. pp_0 , pp_1 , and pp_2 are defined analogously to those in the snark construction, for the exact same relations.

The ledger operator generates $(\tau_3, pp_3) \leftarrow \mathcal{G}(\mathcal{R}_3)$ for relation $\mathcal{R}_3$ defined by:

$$\begin{aligned} & \text{Open}(CK, tk, r, C) \rightarrow 1 \quad \wedge \quad ek = \mathcal{H}_E(e)^{tk} \\ & \wedge \quad \text{tag} = e(ek, \prod_{i=0}^{\ell} G_i^{x_i}) \quad \wedge \quad (x_0, \dots, x_\ell) \in \{0, 1\}^{\ell+1} \quad \wedge \quad \gamma < max. \end{aligned}$$

Bulletproofs are used to enforce bit correctness and the range constraint. The ledger records $(PK, CK, pp_0, pp_1, pp_2, pp_3, max)$.

Protocol execution. The Register, Grant, Submit, Trace, and IncrementEpoch algorithms are again identical to those in Section 4, using the concrete PRFs and relations defined above. Tracing is performed by recomputing all potential tags for a given user and epoch as follows:

$$\text{tag} = e(ek, \prod_{i=0}^{\ell} G_i^{x_i}), \quad (16)$$

for all $\gamma \in [0, max)$, and checking the ledger for matches.

Theorem 2. *The pairing-based instantiations described in this section satisfy the assumptions of Theorem 1. Specifically, we assume the following:*

- *The commitments used to instantiate $\mathbf{G}$ and $\mathbf{G}'$ are one-way functions.*
- *The signature scheme used for credential issuance is existentially unforgeable.*
- *The epoch and tag PRFs $\mathbf{F}_E$ and $\mathbf{F}_T$ satisfy pseudorandom function security.*
- *The proof systems used to instantiate NIZK and SoK satisfy completeness, simulatability, and extractability.*

Under the assumptions of Theorem 2, the protocols described in this section securely realize the ideal functionality $\mathcal{F}_{\text{Trace}}$.

5.3 Experimental Evaluation

Experimental Setup. All experiments were conducted on an Apple M1 Pro laptop with 16GB RAM, 8 physical cores, and one thread per core. Measurements were collected using Go’s built-in benchmarking framework, averaged over the number of iterations taken to reach 1 second total computation.

The SNARK implementation uses the **gnark** backend, which includes optimized multi-pairing routines and assembly-level curve arithmetic. The algebraic construction is implemented as a research prototype without batching, precomputation, or multi-exponentiation optimizations. Consequently, the algebraic benchmarks represent conservative performance estimates.

Hash-based PRF. For the hash-based instantiation, correctness of the PRF is proven using the Groth16 zk-SNARK system [15]. Each proof consists of three group elements. The arithmetic circuit contains 2538 for BN256 and 2584 constraints with BLS12-381. Verification requires 3 pairing evaluations and 1 multi-exponentiation over the public inputs. Benchmark results are summarized in Table 1.

Table 1. Performance of the SNARK-based PRF instantiation.

Operation	Groth16 over BN254	Groth16 over BLS12-381
Setup time (ms)	186	306
Proof time (ms)	11	18
Verification time (ms)	0.92	1.19
Proof size (bytes)	128	192
Constraints	2538	2584
Proving RAM (MB)	26	24
Verification RAM (MB)	22	36

Table 2. Microbenchmarks for the algebraic PRF instantiation.

Operation	BN254	BLS12-381
Epoch proof time (ms)	0.07	0.11
Epoch verification time (ms)	0.11	0.20
Tag proof time (ms)	2.01	3.32
Tag verification time (ms)	4.54	6.33
Proof time (ms)	2.08	3.43
Verification time (ms)	4.65	6.53

Algebraic PRF. We next benchmark the algebraic instantiation. Here, the constraints are enforced via pairing-based Sigma protocols alongside bulletproofs.

Verification of the epoch-key proof requires $2 + 8\ell$ exponentiations in $\mathbb{G}_1$, 1 pairing evaluation, and 1 exponentiation in $\mathbb{G}_T$. Verification of the tag proof additionally requires 4ℓ multi-exponentiations over $\mathbb{G}_2$, 1 pairing evaluation, and 1 exponentiation in $\mathbb{G}_T$. The linear terms in bit-length of the maximum tag index (ℓ) come from use of bulletproofs. We run the benchmarks with $\ell = 10$. Table 2 reports benchmark results.

These results show that the algebraic instantiation achieves proof generation in a few milliseconds and verification in under 10ms even without optimization. While verification is slower than the SNARK-based construction, it remains fast enough for practical deployment, and avoids a trusted setup.

Comparison with Idemix. For context, we compare our SNARK-based instantiation against Idemix, a widely deployed anonymous credential system implemented over BLS12-381¹.

Without tracing, Idemix requires approximately 4ms for signing and 3ms for verification. Our SNARK-based construction incurs additional signing overhead when tracing is enabled but achieves sub-millisecond verification. Since distributed payment systems verify each transaction many times but generate it only once, verification cost dominates overall system throughput. This demon-

¹ <https://github.com/ibm/idemix>

strates that our construction achieves verification costs comparable to widely deployed anonymous credential systems while additionally supporting tracing.

Overall, our results show that both instantiations achieve practical performance. The SNARK-based construction provides extremely fast verification and compact proofs, while the algebraic construction achieves competitive performance without requiring trusted setup. These results support the feasibility of deploying our scheme in real-world systems.

6 Application to Privacy-Preserving Transaction Systems

We describe how the black-box construction of Section 4 can be integrated into a generic distributed ledger-based payment system. We abstract away the implementation of the ledger, including its underlying consensus algorithm, and focus on transaction generation and epoch-bound tagging.

We build from a transaction system that supports publicly verifiable, private transactions of the form $\text{tx} = (\text{payload}, \text{tag}, \pi, \psi)$, with ledger-enforced verification of proofs and signatures of knowledge, and uniqueness of transaction identifiers. To integrate the tagging system into the transaction system, we specify what exactly is contained in **payload**, and how the proofs are extended to enforce the additional constraints needed on the payload values.

The requirements of the transaction system are as follows:

- Balance preservation: no creation of value;
- Unforgeability: only the owner of a token can spend it;
- Double-spending prevention: each token can only be spent once;
- Transaction privacy: values and identities involved are not known.

We now give a simple commitment-based instantiation satisfying these properties and show how epoch-bound tagging is integrated.

6.1 Sender-only transaction tagging

We represent tokens as $\text{coin} = (C, \text{tag})$, where $\text{Commit}(v, r) = g^v h^r$ corresponds to a committed value v . A token owned by user U is $(\text{Commit}(v_{in}, r), \text{tag}_{in})$, where tag_{in} is the epoch-bound tag derived as in Section 4.

A token owned by user U is represented as $(\text{Commit}(v_{in}, r), \text{tag}_{in})$, committing to value v_{in} and tag_{in} is the epoch-bound tag derived as in Section 4.

To transfer value $v_{out} < v_{in}$ from an input coin $(\text{com}_{in}, \text{tag}_{in})$, the sender creates the following:

- A recipient output commitment $\text{com}_{out} = \text{Commit}(v_{out}, r_1)$,
- A change commitment $\text{com}_c = \text{Commit}(v_{in} - v_{out}, r_2)$.

The payload is then formed $\text{payload} = (\text{com}_{in}, \text{com}_{out}, \text{com}_c)$.

The sender proves knowledge of the opening $com_{in} = \text{Commit}(v_{in}, r) = g_{in}^v h^r$, and value preservation, $com_{in} = com_{out} \cdot com_c$. These statements are incorporated into the NIZK of Section 4 by extending $\mathcal{R}_3$ to contain the following:

$$\text{Open}(CK, v_{in}, r, com_{in}) \rightarrow 1 \quad \wedge \quad com_{in} = com_{out} \cdot com_c.$$

For sender-only tagging, the sender computes and attaches tag_γ to the transaction as in Section 4. The complete protocol is given in algorithm 1.

Algorithm 1 Sender-Only Tagging Protocol

Require: User tk_S (user tagging key), current epoch e , input coin $(com_{in}, \text{tag}_{in})$

Ensure: Transaction $\text{tx} = (\text{payload}, \text{tag}_S, \pi_S, \psi_S)$

- 1: Retrieve tk_S and corresponding signing key SK_S
 - 2: Compute $ek_S \leftarrow \mathbf{F}_E(tk_S, e)$
 - 3: Sample $\gamma_S \leftarrow [0, max]$
 - 4: Compute $\text{tag}_S \leftarrow \mathbf{F}_T(ek_S, \gamma_S)$
 - 5: Create recipient output commitment: $com_{out} \leftarrow \text{Commit}(v_{out}, r_1)$
 - 6: Create change commitment: $com_c \leftarrow \text{Commit}(v_{in} - v_{out}, r_2)$
 - 7: Set $\text{payload} \leftarrow (com_{in}, com_{out}, com_c)$
 - 8: Prove knowledge of com_{in} opening: $\text{Open}(CK, v_{in}, r, com_{in}) \rightarrow 1$
 - 9: Prove value preservation: $com_{in} = com_{out} \cdot com_c$
 - 10: Prove correctness of tag_S
 - 11: Form π_S
 - 12: Compute signature of knowledge: $\psi_S \leftarrow \text{DSSign}(SK_S, \text{payload} \parallel \text{tag}_S \parallel \pi_S)$
 - 13: **return** $\text{tx} \leftarrow (\text{payload}, \text{tag}_S, \pi_S, \psi_S)$
-

6.2 Sender and Recipient Tagging

For interactive payment systems, the recipient additionally produces a tag tag_R for the received output commitment com_{out} . The recipient computes:

$$ek_R = \mathbf{F}_E(tk_R, e), \quad \gamma_R \leftarrow [0, max], \quad \text{tag}_R = \mathbf{F}_T(ek_R, \gamma_R),$$

and proves knowledge of the opening of com_{out} and correctness of tag_R .

The sender signs the payload, tag, and proofs, and the recipient signs the output commitment and their tag. The final transaction is then formed as follows:

$$\text{tx} = (\text{payload}, \text{tag}_S, \pi_S, \psi_S, \text{tag}_R, \pi_R, \psi_R), \quad (17)$$

where π_S and π_R are NIZKs proving correctness of the sender's and recipient's tags respectively, and ψ_S and ψ_R are signatures of knowledge from the sender and recipient. The complete sender and recipient tagging protocol is shown in algorithm 2.

Algorithm 2 Sender and Recipient Tagging Protocol

Require: Sender tk_S , Recipient tk_R , current epoch e , input coin (com_{in}, tag_{in})
Ensure: Transaction $tx = (payload, tag_S, \pi_S, \psi_S, tag_R, \pi_R, \psi_R)$

- 1: **Sender:**
- 2: Compute $ek_S \leftarrow F_E(tk_S, e)$
- 3: Sample $\gamma_S \leftarrow [0, max)$
- 4: Compute $tag_S \leftarrow F_T(ek_S, \gamma_S)$
- 5: Create recipient output commitment $com_{out} \leftarrow \text{Commit}(v_{out}, r_1)$
- 6: Create change commitment $com_c \leftarrow \text{Commit}(v_{in} - v_{out}, r_2)$
- 7: Set $payload \leftarrow (com_{in}, com_{out}, com_c)$
- 8: Prove knowledge of com_{in} opening $\text{Open}(CK, v_{in}, r, com_{in}) \rightarrow 1$
- 9: Prove value preservation $com_{in} = com_{out} \cdot com_c$
- 10: Prove correctness of tag_S
- 11: Combine into NIZK π_S
- 12: Compute $\psi_S \leftarrow \text{DSSign}(SK_S, payload, tag_S, \pi_S)$
- 13: Send $(payload, tag_S, \pi_S, \psi_S)$ to recipient.
- 14: **Recipient:**
- 15: Compute $ek_R \leftarrow F_E(tk_R, e)$
- 16: Sample $\gamma_R \leftarrow [0, max)$
- 17: Compute $tag_R \leftarrow F_T(ek_R, \gamma_R)$
- 18: Prove knowledge of com_{out} opening $\text{Open}(CK, v_{out}, r_1, com_{out}) \rightarrow 1$
- 19: Prove correctness of tag_R
- 20: Form π_R
- 21: Compute $\psi_R \leftarrow \text{DSSign}(SK_R, com_{out}, tag_R, \pi_R)$
- 22: **return** $tx \leftarrow (payload, tag_S, \pi_S, \psi_S, tag_R, \pi_R, \psi_R)$

Both the sender-only and sender and recipient models inherit anonymity and per-epoch traceability from the initial properties of the transaction system combined with the tagging scheme pseudorandomness as shown in Theorem 1 and the hiding property of the commitment scheme, while balance preservation, unforgeability, and double-spending resistance follow from the underlying transaction scheme combined with the binding property of the commitment scheme.

7 Token Transfer System

We now build a concrete token transfer system with tracing. We first present a very simple privacy-preserving token scheme, and then show how to integrate our tracing mechanism. Each token is defined by its value and a commitment to its current owner, and transfers are traced via the epoch-based tagging mechanism.

Tokens are represented by tuples $coin = (v, C)$ consisting of the following:

- $v \in \mathbb{Z}_p$, the value of the token,
- C , a commitment to the current owner's public key $S = \mathbf{G}(sk)$, computed as $C = \text{Commit}(CK, S, sn)$ where sn is a secret random value known only to the token holder.

The serial number sn serves two purposes. It is the randomness of the commitment, binding the token to its current owner, and it will be revealed at spend to prevent double-spending, as the ledger maintains a set of all sn values that have been spent. Until the token is spent, sn remains secret to the token holder.

7.1 Token Spending Protocol

We extend the protocol of Section 4 to handle token transfers. The key addition is a new relation for proving knowledge of a token's current owner and the ability to transfer it to a new owner.

Setup During setup, in addition to the parameters $\mathcal{L}[pp]$ from Section 4, the ledger maintains a set of all serial numbers that have been spent. This set prevents double-spending of tokens.

Mint To introduce tokens into the system, a minting authority (or initial registration) creates a token $\text{coin} = (v, C)$ where $C = \text{Commit}(CK, S_0, sn_0)$ for initial owner U_0 with public key S_0 and fresh random serial number sn_0 . The minting authority records the token's commitment C on the ledger. The owner U_0 is privately given the opening (S_0, sn_0) .

Transfer To transfer a token $\text{coin} = (v, com_{in})$ to a new owner U' , the current owner U performs the following:

1. U selects a new random serial number sn' and computes the new commitment $com_{out} = \text{Commit}(CK, S', sn')$ where $S' = \mathbf{G}(sk')$ is the recipient's public key.
2. U generates a NIZK π_{transfer} for a new relation $\mathcal{R}_{\text{coin}}$ that proves knowledge of the opening of com_{in} and correct computation of com_{out} .
3. U obtains the current epoch's tracing key ek_e and computes the transfer tag $\text{tag}_{\text{transfer}} = \mathbf{F}_T(ek_e, \gamma)$ where γ is a fresh counter in the range $[0, max)$.
4. U creates a commitment $C_{\text{payload}} = \text{Commit}(CK, \text{payload})$ where $\text{payload} = (v, com_{in}, com_{out}, sn)$ encodes the transfer details. Here sn is the old serial number being spent.
5. U generates a NIZK π_{tag} proving that the tag $\text{tag}_{\text{transfer}}$ is correctly computed (as in relation $\mathcal{R}_3$ from Section 4).
6. U produces a signature of knowledge $\psi_{\text{transfer}} = \text{SoKSign}(pp_2, \mathbb{X}_2, \mathbb{W}_2, m)$ where $m = (v, com_{in}, com_{out}, sn, C_{\text{tx}}, \text{tag}_{\text{transfer}}, \pi_{\text{tag}})$.
7. The transaction submitted to the ledger is:

$$\text{tx}_{\text{token}} = (v, com_{in}, com_{out}, sn, C_{\text{tx}}, \text{tag}_{\text{transfer}}, \pi_{\text{tag}}, \pi_{\text{transfer}}, \psi_{\text{transfer}}). \quad (18)$$

Token Transfer Relation We define relation $\mathcal{R}_{\text{token}}$ as follows:

$$\mathbb{x}_{\text{token}} = (CK, com_{in}, com_{out}, v), \quad \mathbb{w}_{\text{token}} = (S, sn, S', sn'), \quad (19)$$

such that $\mathcal{R}_{\text{token}}(\mathbb{x}_{\text{token}}, \mathbb{w}_{\text{token}}) = 1$ if and only if the following holds:

$$com_{in} = \text{Commit}(CK, S, sn) \quad \wedge \quad com_{out} = \text{Commit}(CK, S', sn'). \quad (20)$$

This relation proves that the prover knows both the old and new commitments' openings, binding the transfer from one owner to another.

Ledger Verification Upon receiving transaction tx_{token} , the ledger performs the following checks:

1. Verify that sn has not been seen before.
2. Verify that com_{in} corresponds to a token on the ledger.
3. Verify that $\text{tag}_{\text{transfer}}$ has never appeared on the ledger before and that π_{tag} is a valid NIZK proof for relation $\mathcal{R}_3$.
4. Verify that π_{transfer} is a valid NIZK proof for relation $\mathcal{R}_{\text{token}}$.
5. Verify that ψ_{transfer} is a valid signature of knowledge for relation $\mathcal{R}_2$ on message m .

If all checks pass, the ledger:

- Records the new token state: $\mathcal{L}[com_{out}] \leftarrow (v, com_{out})$.
- Marks the old token as spent (optionally archived): the entry $\mathcal{L}[com_{in}]$ is updated or removed.
- Adds sn to the set of spent tokens.
- Records the transaction on the ledger: $\mathcal{L}[\text{tag}_{\text{transfer}}] \leftarrow \text{tx}_{\text{token}}$.

Tracing Token Transfers Given an epoch e_i and the corresponding tracing key ek_i (obtained from the user during audit), an auditor A traces all token transfers by that user during that epoch as follows:

For each transaction $\text{tag}_{\text{transfer}}$ on the ledger that falls within epoch e_i , the auditor retrieves the transaction tx_{token} . By knowledge of the epoch key ek_i , the auditor can verify (without additional interaction) that the $\text{tag}_{\text{transfer}}$ was indeed computed by the user, thereby confirming which tokens the user transferred in that epoch.

More formally, the auditor checks: $\forall 0 \leq \gamma < \max$, compute $\text{tag}_{\gamma}^{(e_i)} = \text{FT}(ek_i, \gamma)$ and retrieve any transaction entries $\mathcal{L}[\text{tag}_{\gamma}^{(e_i)}]$ that match.

7.2 Security Properties

The token transfer system inherits and strengthens the security guarantees of the base scheme:

- No double-spending: The maintenance of the set of spent tokens ensures that each serial number can be seen at most once. An adversary cannot replay a transaction or reuse a token’s serial number.
- Non-repudiation: Each transfer is signed via the signature of knowledge ψ_{transfer} , which proves the spender knew their private key. A user cannot deny having transferred a token.
- Anonymity within an epoch: Within a given epoch, token transfers are unlinkable via the ledger; the tag and commitments reveal nothing about the spender’s identity. Auditors can only learn the spender’s identity (via the epoch key) if explicitly granted access.
- Epoch-restricted tracing: Auditors can only trace tokens transferred in the epochs for which they hold the tracing key. Once an epoch transitions, the epoch key changes and prior tokens cannot be linked to future transfers.
- Owner privacy: The commitment com_{out} hides the recipient’s identity from the ledger; only the spender and recipient know the opening.

7.3 Batch Transfers

The above protocol describes the transfer of a single token per transaction. The scheme naturally extends to batch transfers of multiple tokens in a single transaction. Rather than a single pair (com_{in}, com_{out}) , a batch transaction would include sets of inputs $\{com_{in}^{(j)}\}_{j=1}^m$ and outputs $\{com_{out}^{(k)}\}_{k=1}^n$, with corresponding serial numbers and proofs. The ledger would verify that: (1) all input serial numbers are fresh, (2) all tags are fresh, (3) all NIZK proofs are valid, and (4) the total value in equals the total value out (conservation of value).

This extension does not fundamentally change the security properties; we defer a detailed treatment to future work.

Theorem 3. *Assuming existential unforgeability of DS, pseudo-randomness of $\mathbf{F}_E$ and $\mathbf{F}_T$, binding and hiding of Commit, one-wayness of $\mathbf{G}$ and $\mathbf{G}'$, and security of NIZK and SoK, the token transfer system described above securely realizes anonymous, auditable, and double-spend-free token transfers in the presence of a static, polynomial-time adversary.*

The proof sketch is as follows: non-repudiation and double-spend-free property follow from the non-forgeability of the signature scheme and the use of unique serial numbers; anonymity within an epoch follows from the unlinkability of tags within an epoch and the hiding property of commitments; epoch-based tracing follows from the pseudo-randomness of the PRF (the auditor can only compute tags for epochs they hold keys for); and owner privacy follows from the hiding of the commitment scheme.

8 Conclusion

We introduced a new construction for efficient tracing in privacy-preserving payment systems. Our approach enables an authorized auditor to identify all trans-

actions associated with a target user within a given epoch, while preserving anonymity for all other users.

Unlike prior work, our construction does not rely on encrypted user identifiers or sequential counters. This allows users to generate transactions in parallel without coordination and avoids the scalability limitations of existing approaches.

We presented two concrete instantiations: a SNARK-based construction that achieves compact proofs and sub-millisecond verification, and an algebraic construction that avoids trusted setup while maintaining practical performance. Our experimental results demonstrate that both instantiations are efficient and suitable for deployment.

Our work shows that efficient, non-interactive, and scalable tracing can be achieved without sacrificing concurrency or requiring trusted setup. This provides a practical foundation for regulatory-compliant privacy-preserving payment systems.

Future work. An interesting direction for future work is optimizing the algebraic construction, as well as integrating the scheme into full payment protocols and evaluating performance in large-scale deployments.

References

1. Central bank digital currency data use and privacy protection. Imf fintech note 2024/004, international monetary fund, washington, dc, International Monetary Fund (2024), <https://www.imf.org/-/media/files/publications/ftn063/2024/english/ftnea2024004.pdf>
2. Allen, S., Čapkun, S., Eyal, I., Fanti, G., Ford, B.A., Grimmelmann, J., Juels, A., Kostiainen, K., Meiklejohn, S., Miller, A.: Design choices for central bank digital currency: Policy and technical considerations. Tech. rep., National Bureau of Economic Research (2020)
3. Androuraki, E., Camenisch, J., Caro, A.D., Dubovitskaya, M., Elkhayaoui, K., Tackmann, B.: Privacy-preserving auditable token payments in a permissioned blockchain system. In: AFT '20: 2nd ACM Conference on Advances in Financial Technologies, New York, NY, USA, October 21-23, 2020. pp. 255–267. ACM (2020). <https://doi.org/10.1145/3419614.3423259>
4. Bank for International Settlements: Central bank digital currencies: System design and privacy/regulatory trade-offs. BIS Papers OTHP 88, Bank for International Settlements (2022), https://www.bis.org/publ/othp88/system_design.pdf
5. Barki, A., Gouget, A.: Achieving privacy and accountability in traceable digital currency. Cryptology ePrint Archive (2020)
6. Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from bitcoin. In: 2014 IEEE Symposium on Security and Privacy, SP 2014, Berkeley, CA, USA, May 18-21, 2014. pp. 459–474. IEEE Computer Society (2014). <https://doi.org/10.1109/SP.2014.36>, <https://doi.org/10.1109/SP.2014.36>
7. Béres, F., Seres, I.A., Benczúr, A.A., Quintyne-Collins, M.: Blockchain is watching you: Profiling and deanonymizing ethereum users. In: 2021 IEEE international

- conference on decentralized applications and infrastructures (DAPPS). pp. 69–78. IEEE (2021)
8. Camenisch, J., Hohenberger, S., Lysyanskaya, A.: Balancing accountability and privacy using e-cash (extended abstract). In: Prisco, R.D., Yung, M. (eds.) *Security and Cryptography for Networks*, 5th International Conference, SCN 2006, Maiori, Italy, September 6–8, 2006, Proceedings. *Lecture Notes in Computer Science*, vol. 4116, pp. 141–155. Springer (2006). <https://doi.org/10.1007/11832072>“10, https://doi.org/10.1007/11832072_10
 9. Camenisch, J., Lehmann, A.: Privacy-preserving user-auditable pseudonym systems. In: 2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26–28, 2017. pp. 269–284. IEEE (2017). <https://doi.org/10.1109/EUROSP.2017.36>, <https://doi.org/10.1109/EuroSP.2017.36>
 10. Chen, Y., Ma, X., Tang, C., Au, M.H.: PGC: decentralized confidential payment system with auditability. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) *Computer Security - ESORICS 2020 - 25th European Symposium on Research in Computer Security*, ESORICS 2020, Guildford, UK, September 14–18, 2020, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 12308, pp. 591–610. Springer (2020). <https://doi.org/10.1007/978-3-030-58951-6>“29, https://doi.org/10.1007/978-3-030-58951-6_29
 11. Connolly, A., Deschamps, J., Lafourcade, P., Perez-Kempner, O.: Protego: Efficient, revocable and auditable anonymous credentials with applications to hyperledger fabric. In: Isobe, T., Sarkar, S. (eds.) *Progress in Cryptology - INDOCRYPT 2022 - 23rd International Conference on Cryptology in India*, Kolkata, India, December 11–14, 2022, Proceedings. *Lecture Notes in Computer Science*, vol. 13774, pp. 249–271. Springer (2022). <https://doi.org/10.1007/978-3-031-22912-1>“11, https://doi.org/10.1007/978-3-031-22912-1_11
 12. Fauzi, P., Meiklejohn, S., Mercer, R., Orlandi, C.: Quisquis: A new design for anonymous cryptocurrencies. In: Galbraith, S.D., Moriai, S. (eds.) *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security*, Kobe, Japan, December 8–12, 2019, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 11921, pp. 649–678. Springer (2019). <https://doi.org/10.1007/978-3-030-34578-5>“23, https://doi.org/10.1007/978-3-030-34578-5_23
 13. Garman, C., Green, M., Miers, I.: Accountable privacy for decentralized anonymous payments. In: Grossklags, J., Preneel, B. (eds.) *Financial Cryptography and Data Security - 20th International Conference, FC 2016, Christ Church, Barbados, February 22–26, 2016, Revised Selected Papers*. *Lecture Notes in Computer Science*, vol. 9603, pp. 81–98. Springer (2016). <https://doi.org/10.1007/978-3-662-54970-4>“5, https://doi.org/10.1007/978-3-662-54970-4_5
 14. Goldwasser, S., Micali, S., Rackoff, C.: The knowledge complexity of interactive proof-systems (extended abstract). In: Sedgewick, R. (ed.) *Proceedings of the 17th Annual ACM Symposium on Theory of Computing*, May 6–8, 1985, Providence, Rhode Island, USA. pp. 291–304. ACM (1985). <https://doi.org/10.1145/22145.22178>, <https://doi.org/10.1145/22145.22178>
 15. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J. (eds.) *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, Austria, May 8–12, 2016, Proceedings, Part II. *Lecture Notes in Computer*

- Science, vol. 9666, pp. 305–326. Springer (2016). https://doi.org/10.1007/978-3-662-49896-5_11, https://doi.org/10.1007/978-3-662-49896-5_11
16. Hub, B.I.: Project mbridge: Connecting economies through cbdc. <https://www.bis.org/publ/othp59.htm> (2022)
 17. Jinze, E.: First look: China’s central bank digital currency. <https://research.binance.com/analysis/china-cbdc> (2019)
 18. Kappos, G., Yousaf, H., Maller, M., Meiklejohn, S.: An empirical analysis of anonymity in zcash. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 463–477 (2018)
 19. Kiayias, A., Kohlweiss, M., Sarencheh, A.: Peredi: Privacy-enhanced, regulated and distributed central bank digital currencies. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7–11, 2022. pp. 1739–1752. ACM (2022). <https://doi.org/10.1145/3548606.3560707>, <https://doi.org/10.1145/3548606.3560707>
 20. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. Satoshi Nakamoto (2008)
 21. Narula, N., Vasquez, W., Virza, M.: zkledger: Privacy-preserving auditing for distributed ledgers. In: Banerjee, S., Seshan, S. (eds.) 15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018, Renton, WA, USA, April 9–11, 2018. pp. 65–80. USENIX Association (2018), <https://www.usenix.org/conference/nsdi18/presentation/narula>
 22. Noether, S.: Ring signature confidential transactions for monero. IACR Cryptol. ePrint Arch. p. 1098 (2015), <http://eprint.iacr.org/2015/1098>
 23. OECD: Central bank digital currencies (CBDCs) and democratic values (2023). <https://doi.org/https://doi.org/10.1787/f3e70f1f-en>, https://www.oecd.org/content/dam/oecd/en/publications/reports/2023/07/central-bank-digital-currencies-cbdc-and-democratic-values_81f16e0c/f3e70f1f-en.pdf
 24. Papadoulis, G., Balla, D., Grontas, P., Pagourtzis, A.: AQQUA: augmenting quisquis with auditability. In: Fischlin, M., Moonsamy, V. (eds.) Applied Cryptography and Network Security - 23rd International Conference, ACNS 2025, Munich, Germany, June 23–26, 2025, Proceedings, Part II. Lecture Notes in Computer Science, vol. 15826, pp. 283–312. Springer (2025). https://doi.org/10.1007/978-3-031-95764-2_12, https://doi.org/10.1007/978-3-031-95764-2_12
 25. Sarencheh, A., Kiayias, A., Kohlweiss, M.: Parscoin: A privacy-preserving, auditable, and regulation-friendly stablecoin. In: Kohlweiss, M., Pietro, R.D., Beresford, A.R. (eds.) Cryptology and Network Security - 23rd International Conference, CANS 2024, Cambridge, UK, September 24–27, 2024, Proceedings, Part I. Lecture Notes in Computer Science, vol. 14905, pp. 289–313. Springer (2024). https://doi.org/10.1007/978-981-97-8013-6_13, https://doi.org/10.1007/978-981-97-8013-6_13
 26. Wüst, K., Kostiaainen, K., Delius, N., Capkun, S.: Platypus: A central bank digital currency with unlinkable transactions and privacy-preserving regulation. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7–11, 2022. pp. 2947–2960. ACM (2022). <https://doi.org/10.1145/3548606.3560617>, <https://doi.org/10.1145/3548606.3560617>

9 Preliminaries

9.1 Computational Assumptions

Decisional Diffie-Hellman (DDH). The DDH assumption holds in $\mathbb{G}_1$ if no PPT adversary can distinguish (g_1^a, g_1^b, g_1^{ab}) from (g_1^a, g_1^b, g_1^c) for uniformly random $a, b, c \in \mathbb{Z}_p$.

Bilinear Decisional Diffie-Hellman (BDDH). The BDDH assumption holds if no PPT adversary can distinguish $(g_1, g_1^a, g_1^b, g_2, g_2^c, e(g_1, g_2)^{abc})$ from $(g_1, g_1^a, g_1^b, g_2, g_2^c, T)$ for uniformly random $a, b, c \in \mathbb{Z}_p$ and random $T \in \mathbb{G}_T$.

9.2 Commitment Schemes

A commitment scheme is secure if it satisfies the properties of correctness and binding.

Correctness. A commitment scheme is correct if:

$$\Pr [\text{Open}(CK, m, \rho, C) \rightarrow 1 \mid CK \leftarrow \text{ComKeygen}(1^\kappa) \wedge (\rho, C) \leftarrow \text{Commit}(CK, m)] = 1.$$

Binding. Let ε denote a negligible function in security parameter κ . A commitment scheme is binding, if for all efficient adversaries $\mathcal{A}$, the following holds:

$$\Pr \left[m_0 \neq m_1 \wedge \text{Open}(CK, m_0, \rho_0, C) \rightarrow 1 \mid \begin{array}{l} CK \leftarrow \text{ComKeygen}(1^\kappa) \\ \wedge \text{Open}(CK, m_1, \rho_1, C) \rightarrow 1 \end{array} \wedge (m_0, \rho_0, m_1, \rho_1, C) \leftarrow \mathcal{A}(CK) \right] \leq \varepsilon.$$

Finally, a commitment scheme is said to be **hiding** iff, for all efficient adversaries $\mathcal{A}$, we have:

$$\Pr \left[b = b^* \mid \begin{array}{l} CK \leftarrow \text{ComKeygen}(1^\kappa) \wedge (m_0, m_1) \leftarrow \mathcal{A}(CK) \\ \wedge b \in \{0, 1\} \wedge (\star, C_b) \leftarrow \text{Commit}(CK, m_b) \\ \wedge b^* \leftarrow \mathcal{A}(CK, m_0, m_1, C_b) \end{array} \right] - 1/2 \leq \varepsilon.$$

9.3 Pedersen Commitments

The Pedersen commitment scheme is used to commit to a message or vector of messages without leaking any information about the committed vector. It consists of the following algorithms:

- **ComKeygen** $(1^\kappa, n)$ On input of security parameter κ and integer n , **ComKeygen** outputs commitment key CK .
- **Commit** (CK, m, r) On input of commitment key CK , message $m \in \mathbb{Z}_p$, and randomness $r \in \mathbb{Z}_p$, **Commit** computes $com = CK^m P^r$.
- **Open** (CK, com, m, r) On input of commitment key CK , commitment com , message $m \in \mathbb{Z}_p$, and randomness r , **Open** outputs 1 if $com = CK^m P^r$; otherwise, it outputs 0.

Pedersen commitments are unconditionally hiding and computationally binding under the discrete logarithm assumption.

9.4 BBS+ Signatures

The BBS+ signature scheme allows a party to sign a vector of messages of a predefined length n , and is defined by three algorithms as follows:

- $\text{BBSKeyGen}(1^\kappa, n) \rightarrow (SK, PK)$: On input security parameter κ and size n of the vectors of messages to be signed, BBSKeyGen first samples generators $P, P_1, \dots, P_n \xleftarrow{\$} \mathbb{G}_1$ and $Q \xleftarrow{\$} \mathbb{G}_2$, followed by sampling $z \xleftarrow{\$} \mathbb{Z}_p$. Next, BBSKeyGen computes $Z = Q^z$, and outputs secret key $SK = z$ and public key $PK = (P, P_1, \dots, P_n, Q, Z)$.
- $\text{BBSSign}(SK, \mathbf{m}) \rightarrow \sigma$: On input secret key $SK = z$ and vector of n messages $\mathbf{m} = (m_1, \dots, m_n) \in \mathbb{Z}_p^n$, BBSSign randomly samples $e, r \xleftarrow{\$} \mathbb{Z}_p$ such that $e + z \neq 0$, computes $E = (P^r \prod_{i=1}^n P_i^{m_i})^{\frac{1}{z+e}}$, and finally, outputs signature $\sigma = (E, e, r)$.
- $\text{BBSVerify}(PK, \mathbf{m}, \sigma) \rightarrow 0/1$: On input public key PK , vector of n messages $\mathbf{m}$, and signature $\sigma = (E, e, r)$, BBSVerify outputs 1 if $e(E, Q^e Z) = e(P^r \prod_{i=1}^n P_i^{m_i}, Q)$, indicating, hence, that the signature is valid. Otherwise, BBSVerify outputs 0, signifying that the signature as invalid.

BBS+ signatures are *existentially unforgeable* under the q -Strong Diffie-Hellman (q -SDH) assumption.

Definition 1. *q -Strong Diffie-Hellman assumption.* Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ be bilinear groups of prime order p with generators $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$. The q -Strong Diffie-Hellman (q -SDH) assumption states that, given the following:

$$(P, P^x, P^{x^2}, \dots, P^{x^q}, Q), \quad (21)$$

for a randomly chosen $x \xleftarrow{\$} \mathbb{Z}_p$, it is computationally infeasible for any probabilistic polynomial-time adversary to output a pair $(c, P^{1/(x+c)})$ for any $c \in \mathbb{Z}_p \setminus \{-x\}$ with non-negligible probability.

A proof of knowledge of a BBS+ signature can be formed as follows. Let witness $\mathbf{w} = (E, e, r, m_1, \dots, m_n, \rho)$ and instance $\mathbf{x} = (G, H, C, P, P_1, \dots, P_n, Q, Z)$. Then $\mathcal{R}(\mathbf{w}, \mathbf{x}) = 1$ if and only if the following holds:

$$C = G^{m_n} H^\rho \wedge e(E, Q^e Z) = e(P^r \prod_{i=1}^n P_i^{m_i}, Q). \quad (22)$$

9.5 PRF Security of $\mathbf{F}_E$

Lemma 1. *Under the DDH assumption in $\mathbb{G}_1$ and modeling $\mathcal{H}_E$ as a random oracle, the function $\mathbf{F}_E(tk, e) = \mathcal{H}_E(e)^{tk}$ is a secure pseudorandom function.*

Proof. We show that no polynomial-time adversary can distinguish $\mathbf{F}_E(tk, \cdot)$ from a random function with non-negligible advantage.

Let $\mathcal{A}$ be an adversary with oracle access to either $\mathbf{F}_E(tk, \cdot)$ or a random function $R : \{0, 1\}^* \rightarrow \mathbb{G}_1$. We construct a simulator $\mathcal{S}$ that breaks DDH in $\mathbb{G}_1$ if $\mathcal{A}$ distinguishes the two cases.

$\mathcal{S}$ receives a DDH challenge (g, g^a, g^b, Z) where $Z = g^{ab}$ (DDH) or $Z \leftarrow \mathbb{G}_1$ (random). It does the following:

- $tk^* = a$ (unknown to $\mathcal{S}$),
- For each query e_i , program $\mathcal{H}_E(e_i) = g^{r_i}$ for fresh random $r_i \leftarrow \mathbb{Z}_p$,
- Answer the i th query with $Y_i = (g^a)^{r_i} = g^{a \cdot r_i}$.

If $Z = g^{ab}$, then $Y_i = g^{a \cdot r_i} = \mathcal{H}_E(e_i)^{tk^*}$ are correctly distributed as $\mathbf{F}_E(tk^*, e_i)$.

If $Z \leftarrow \mathbb{G}_1$, the random oracle means that each $\mathcal{H}_E(e_i)$ is uniformly random and independent. Since $tk^* = a$ is random and unknown, each response $Y_i = g^{a \cdot r_i}$ is independent and uniformly random in $\mathbb{G}_1$.

We then have the following advantage:

$$\text{Adv}[\mathcal{A}_F] \leq \text{Adv}[\mathcal{S}_{\text{DDH}}] + \frac{q^2}{2|\mathbb{Z}_p|},$$

where q is the number of queries and the second term accounts for collision probability in random oracle programming.

9.6 PRF Security of $\mathbf{F}_T$

Lemma 2. *Under the BDDH assumption in bilinear groups, the function $\mathbf{F}_T(ek, \gamma) = e(ek, \prod_{i=0}^{\ell} G_i^{x_i})$ is a secure pseudorandom function for well-formed inputs γ .*

Proof. Let γ have bitwise decomposition $(x_0, \dots, x_\ell)$ and let $ek = g_1^\alpha$ for some secret $\alpha \in \mathbb{Z}_p$.

Given BDDH challenge $(g_1, g_1^a, g_2, g_2^b, Z)$ where $Z = e(g_1, g_2)^{ab}$ (BDDH) or $Z \leftarrow \mathbb{G}_T$, we do the following:

- $ek^* = g_1^a$ (so $\alpha = a$),
- For query γ with bits $(x_0, \dots, x_\ell)$, set $H = \prod_{i=0}^{\ell} G_i^{x_i}$ and program this as $g_2^{r_\gamma}$ for fresh random r_γ ,
- Return $Y = e(g_1^a, g_2^{r_\gamma}) = e(g_1, g_2)^{a \cdot r_\gamma}$.

If $Z = e(g_1, g_2)^{ab}$, then outputs are correctly distributed as $e(ek^*, H) = e(g_1^a, g_2^{r_\gamma})$.

If $Z \leftarrow \mathbb{G}_T$, since a and each r_γ are independent random values, each $Y = e(g_1, g_2)^{a \cdot r_\gamma}$ is uniformly random in $\mathbb{G}_T$.

We then have the following advantage:

$$\text{Adv}[\mathcal{A}_F] \leq \text{Adv}[\mathcal{S}_{\text{BDDH}}] + \frac{q \cdot 2^\ell}{|\mathbb{G}_T|}.$$

10 Proofs of Knowledge and Bulletproofs

Here we specify the exact statements proven in the pairing-based instantiation described in Section 5.2, together with the concrete Sigma protocols and Bulletproofs used to realize them. All proofs are made non-interactive using the Fiat-Shamir transform.

10.1 Proofs of Knowledge of BBS+ Signatures

Let $I \subseteq \{1, \dots, \ell\}$ be the set of disclosed indices and $J = \{1, \dots, \ell\} \setminus I$. The public input is $(pp, X_2, (m_i)_{i \in I})$, and the witness is $(A, e, s, (m_i)_{i \in J})$ such that $\sigma = (A, e, s)$ is a valid BBS+ signature.

Prover: Sample $r_1, r_2 \xleftarrow{\$} \mathbb{Z}_p^*$ and compute the following:

$$\bar{A} = A^{r_1/r_2}, \quad D = (g_1 g_2^s \prod_{i=1}^{\ell} h_i^{m_i})^{1/r_2}, \quad \bar{B} = D^{r_1} \bar{A}^{-e}. \quad (23)$$

Sample $\alpha, \beta, \gamma \xleftarrow{\$} \mathbb{Z}_p$ and $\delta_i \xleftarrow{\$} \mathbb{Z}_p$ for $i \in I$, and compute the following:

$$U_1 = D^\alpha \bar{A}^\beta, \quad U_2 = D^\gamma \prod_{i \in I} h_i^{\delta_i}. \quad (24)$$

Send $(\bar{A}, \bar{B}, D, U_1, U_2)$.

Verifier: Sample and send challenge $c \leftarrow \mathbb{Z}_p$.

Prover: Compute the following:

$$s_1 = \alpha + r_1 c, \quad t = \beta - e c, \quad z = \gamma + r_2 c, \quad u_i = \delta_i - m_i c \quad \forall i \in I. \quad (25)$$

Send $(s_1, t, z, (u_i)_{i \in I})$.

Verifier: Accept if and only if the following verify:

$$e(\bar{A}, X_2) = e(\bar{B}, g_2) \quad \wedge \quad U_1 \cdot \bar{B}^c = D^{s_1} \bar{A}^t \quad \wedge \quad U_2 \cdot (g_1 \prod_{i \in J} h_i^{m_i})^c = D^z \prod_{i \in I} h_i^{u_i}. \quad (26)$$

10.2 Pairing Product Proofs

We use Sigma protocols to prove pairing product equations of the following form:

$$\prod_{j=1}^m e(A_j, B_j(x_1, \dots, x_n)) = 1_{G_T}, \quad (27)$$

where $A_j \in \mathbb{G}_1$ are public group elements and each B_j is a product of public generators in $\mathbb{G}_2$ to the power of linear equations of the witness elements $x_1, \dots, x_n \in \mathbb{Z}_p$.

These proofs do not require a common reference string. As our relations can be stated in simple pairing products, we can rely on standard Sigma protocols generalized to bilinear groups. With $x_1, \dots, x_n$ as the witness, the protocol is defined as follows:

Prover: Sample $r_1, \dots, r_n \xleftarrow{\$} \mathbb{Z}_p$ and compute the following:

$$T = \prod_{j=1}^m e(A_j, B_j(r_1, \dots, r_n)).$$

Send T .

Verifier: Sample and send $c \xleftarrow{\$} \mathbb{Z}_p$.

Prover: For each $i \in \{1, \dots, n\}$, compute $s_i = r_i + cx_i$ and send $(s_1, \dots, s_n)$.

Verifier: Accept if and only if the following verifies:

$$\prod_{j=1}^m e(A_j, B_j(s_1, \dots, s_n)) = T \cdot \left(\prod_{j=1}^m e(A_j, B_j(x_1, \dots, x_n)) \right)^c.$$

10.3 Proof of Epoch Key Correctness

For a fixed epoch e , the user computes the following:

$$ek = \mathcal{H}_E(e)^{tk} \in \mathbb{G}_1, \quad C = g_G^{tk} g_H^r \in \mathbb{G}_1.$$

Epoch key PRF correctness. The prover proves knowledge of $tk \in \mathbb{Z}_p$ such that $ek = \mathcal{H}_E(e)^{tk}$.

Prover: Sample $r \xleftarrow{\$} \mathbb{Z}_p$ and compute and send $R = \mathcal{H}_E(e)^r$.

Verifier: Sample and send challenge $c \xleftarrow{\$} \mathbb{Z}_p$.

Prover: Compute and send $s = r + ctk$.

Verifier: Accept if and only if the following holds:

$$\mathcal{H}_E(e)^s = R \cdot ek^c.$$

Commitment correctness. The prover proves knowledge of $(tk, r) \in \mathbb{Z}_p^2$ such that $C = g_G^{tk} g_H^r$.

Prover: Sample $r_1, r_2 \xleftarrow{\$} \mathbb{Z}_p$ and compute and send $R = g_G^{r_1} g_H^{r_2}$.

Verifier: Sample and send challenge $c \xleftarrow{\$} \mathbb{Z}_p$.

Prover: Compute and send $s_1 = r_1 + ctk$ and $s_2 = r_2 + cr$.

Verifier: Accept if and only if the following holds:

$$g_G^{s_1} g_H^{s_2} = R \cdot C^c.$$

The same witness tk is reused across both proofs, ensuring consistency between the epoch key ek and the commitment C .

10.4 Proof of Transaction Tag Correctness

For a transaction index $\gamma \in [0, max)$ with bit decomposition $(x_0, \dots, x_\ell)$, the transaction tag is computed as follows:

$$\text{tag}_\gamma = e(ek, \prod_{i=0}^{\ell} G_i^{x_i}) \in \mathbb{G}_T.$$

The witness is $(x_0, \dots, x_\ell)$, and the public input is $(ek, G_0, \dots, G_\ell, \text{tag}_\gamma)$. The prover shows correctness of the tag computation by proving the following pairing product equation:

$$e(ek, \prod_{i=0}^{\ell} G_i^{x_i}) \cdot \text{tag}_\gamma^{-1} = 1_{\mathbb{G}_T}.$$

Prover: Sample $r_0, \dots, r_\ell \xleftarrow{\$} \mathbb{Z}_p$ and compute the following:

$$T = e(ek, \prod_{i=0}^{\ell} G_i^{r_i}).$$

Send T .

Verifier: Sample and send challenge $c \xleftarrow{\$} \mathbb{Z}_p$.

Prover: For each $i \in \{0, \dots, \ell\}$, compute $s_i = r_i + cx_i$, and send $(s_0, \dots, s_\ell)$.

Verifier: Accept if and only if the following holds:

$$e(ek, \prod_{i=0}^{\ell} G_i^{s_i}) = T \cdot \text{tag}_\gamma^c.$$

This proof enforces correctness of the algebraic computation of the transaction tag but does not enforce that the values x_i are bits or that $\gamma \in [0, max)$. These properties are enforced separately using Bulletproofs.

10.5 Bulletproofs

To ensure that the transaction index γ is well formed, the prover uses Bulletproofs over Pedersen commitments.

For each $i \in \{0, \dots, \ell\}$, the prover computes commitment $C_i = g^{x_i} h^{r_i}$. The prover generates a Bulletproof showing $x_i \in \{0, 1\} \ \forall i$ and $\gamma = \sum_{i=0}^{\ell} 2^i x_i$, overall proving that $0 \leq \gamma < max$. The Bulletproof consists of the commitments $(C_0, \dots, C_\ell)$, and a logarithmic sized proof. The challenges are derived through the Fiat-Shamir transform.

11 Security argument for Theorem 1

We prove security of Theorem 1 via real/ideal world indistinguishability. We define a sequence of games, starting from the real execution of the protocol and ending in an execution that is equivalent to the ideal functionality $\mathcal{F}_{\text{Trace}}$. Each game is computationally indistinguishable from the previous one. Throughout the proof, NIZK and SoK systems are treated as black boxes, and we rely on simulatability and extraction properties.

Let $\mathcal{A}$ be a polynomial-time adversary interacting with the real protocol. We construct a simulator $\mathcal{S}$ interacting with the ideal functionality $\mathcal{F}_{\text{Trace}}$, and show that no environment can distinguish the two executions.

The simulator maintains the following tables:

- $\text{reg}[S] = (\sigma, tk)$, logging registered users, where $S = \mathbf{G}(sk)$ serves as the user identity,
- $\text{perm}[(A, S, e)]$, logging tracing permissions,
- $\text{txns}[(S, e)]$, logging submitted transactions.

These tables are analogous to reg , perm , and txns maintained by $\mathcal{F}_{\text{Trace}}$.

Game 0. This is the real execution of the protocol.

Game 1. This game is identical to Game 0, except that the simulator $\mathcal{S}$ now simulates all NIZK and SoKs produced by honest parties using the simulators of the respective proof systems. When an honest user produces a proof for relation $\mathcal{R}_1$, $\mathcal{R}_2$, or $\mathcal{R}_3$, the simulator generates a simulated proof using the public instance and not using the witness.

For corrupted parties, $\mathcal{S}$ verifies proofs normally and applies the extractors of the NIZK and SoK systems when a proof is accepted from a party controlled by the adversary.

From the adversary's perspective, the simulated proofs are indistinguishable from real proofs due to the simulation property of the NIZK and SoK systems. Therefore, Game 0 and Game 1 are computationally indistinguishable.

Game 2. This game is identical to Game 1, except that $\mathcal{S}$ now also handles **Submit** calls on behalf of honest users. For each honest submission during epoch e , instead of computing the tag as follows:

$$\text{tag}_\gamma = \mathbf{F}_\top(ek, \gamma),$$

the simulator samples a uniformly random tag tag^* from the tag space and simulates the corresponding NIZK for relation $\mathcal{R}_3$ and SoK for relation $\mathcal{R}_2$. The simulator records that tag tag^* corresponds to S and epoch e by storing it in $\text{txns}[(S, e)]$.

From the adversary's perspective, other users' tags in the real protocol are outputs of the pseudorandom function $\mathbf{F}_\top$ evaluated on a secret key ek unknown to the adversary. By pseudorandomness of $\mathbf{F}_\top$, the distributions of tag_γ and tag^*

are computationally indistinguishable from one another. The simulated NIZKs and SoKs are indistinguishable from real ones due to the simulation property of both systems. Therefore the adversary's view here in Game 2 is computationally indistinguishable from Game 1.

Now we consider transactions submitted by corrupted users.

When the adversary submits an accepting transaction containing $\pi_{1,\gamma}$ for relation $\mathcal{R}_3$ and $\psi_{2,\gamma}$ for relation $\mathcal{R}_2$, simulation extractability of the two systems guarantees that $\mathcal{S}$ can extract a witness $(sk, tk, \sigma, ek, \gamma)$, such that the following properties hold:

$$\begin{aligned} \text{DSVerify}(PK, \mathbf{G}(sk) \parallel \mathbf{G}'(tk), \sigma) = 1 \quad \wedge \quad \text{Open}(CK, tk, \rho, C) = 1, \\ ek = \mathbf{F}_E(tk, e) \quad \wedge \quad \text{tag} = \mathbf{F}_T(ek, \gamma) \quad \wedge \quad 0 \leq \gamma < \text{max}. \end{aligned}$$

The simulator identifies the submitting user as $S = \mathbf{G}(sk)$ and records the entry in $\text{txns}[(S, e)]$, matching the behaviour of the functionality exactly.

Game 3. This game is identical to Game 2, except $\mathcal{S}$ now also handles **Grant** calls. For honest users, instead of computing the real epoch key $ek = \mathbf{F}_E(tk, e)$, the simulator samples a uniformly random key ek^* and simulates the NIZK and SoKs. The simulator then logs $\text{perm}[(A, S, e)] = 1$.

From the adversary's perspective, real epoch keys are outputs of pseudo-random function $\mathbf{F}_E$ on a secret input tk . By pseudorandomness of $\mathbf{F}_E$, these values are indistinguishable from random. Therefore Game 3 and Game 2 are indistinguishable.

For corrupted users producing grant proofs, simulation extractability allows $\mathcal{S}$ to extract witness tk satisfying $ek = \mathbf{F}_E(tk, e)$, which means that permissions correspond exactly to the functionality behaviour.

Game 4. This game is identical to Game 3, except $\mathcal{S}$ now also handles **Trace** calls. Upon receiving (Trace, S, e) , the simulator returns the set $\text{txns}[(S, e)]$ which it previously recorded. In the real protocol, the auditor computes all tags using ek and retrieves exactly the same transactions. Thus Game 4 is identical to Game 3.

Game 5. This game is identical to Game 4, except $\mathcal{S}$ now also handles **MoveEpoch** calls. Epoch updates are public and identical in both worlds. Thus Game 5 is identical to Game 4.

Since each transition is computationally indistinguishable, the real and ideal executions are indistinguishable.